

Opening

Good morning everyone.

I am Varada Inamdar, and today I will present our work titled "Iterative and Hardware-Reusable AES-128 Architecture for FPGA and IoT Applications."

This work focuses on designing a low-power AES-128 encryption architecture suitable for FPGA-based embedded and IoT systems, where area and energy consumption are often more important than maximum throughput.

Introduction

Secure communication is now essential in IoT and edge devices. These systems collect and transmit sensitive data, such as sensor readings, health information, control signals, or industrial measurements. AES, or Advanced Encryption Standard, is widely used for securing such data because it is standardized, reliable, and cryptographically strong.

However, when AES is implemented in hardware, many designs use parallel or fully unrolled round architectures. These designs can provide high throughput, but they also activate many logic blocks at the same time. This increases switching activity, area, and dynamic power consumption.

For small IoT nodes, this becomes a serious limitation because these systems usually operate with limited power and hardware resources.

Motivation

The motivation of this project comes from the dynamic power equation:

$$P_{\text{dyn}} = \alpha CL V^2 f$$

Here, α represents switching activity, CL is load capacitance, V is supply voltage, and f is clock frequency.

In this work, we mainly target the activity factor α . Instead of reducing voltage or frequency, we reduce unnecessary internal switching by reusing hardware. The key idea is simple: if fewer blocks toggle at the same time, dynamic power reduces naturally.

So the main objective is to design an AES-128 architecture that uses less hardware, reduces switching activity, and still performs correct encryption.

Proposed Architecture

The proposed design is an iterative AES-128 architecture.

In a conventional parallel AES design, multiple round units or transformation blocks may be active together. In our design, we reuse a single datapath across AES rounds.

The datapath contains the main AES transformations: SubBytes, ShiftRows, MixColumns, AddRoundKey, and Key Expansion. Instead of duplicating these blocks for every round, the same hardware is used sequentially under finite state machine control.

AES-128 requires an initial AddRoundKey operation, followed by 10 rounds. Rounds 1 to 9 include SubBytes, ShiftRows, MixColumns, and AddRoundKey. The final round excludes MixColumns as required by the AES standard.

This sequential reuse reduces duplicate logic and lowers simultaneous transitions inside the datapath.

Methodology

The design is implemented at RTL level using SystemVerilog.

At the start, the plaintext and key are loaded. The initial state is generated using:

$$S0 = P \text{ XOR } K0$$

where P is the plaintext and K0 is the initial key.

Then the state passes through the AES transformations. SubBytes performs nonlinear byte substitution using the Rijndael S-box. ShiftRows rotates rows of the AES state matrix. MixColumns performs finite-field matrix multiplication to provide diffusion. AddRoundKey combines the state with the generated round key.

The key expansion block generates the next round key dynamically. This avoids storing all round keys separately and supports the iterative design style.

The control flow is managed by an FSM and a round counter. The FSM activates only the required block at each step. This is what makes the architecture hardware-reusable and low-power.

The uniqueness of this work is that power reduction is considered at the architectural level itself. Instead of adding power optimization only after the AES design is complete, the datapath is designed from the beginning to minimize active hardware per cycle.

FSM-Controlled Architecture

The FSM diagram shows two important sequential blocks.

The first is the sequential MixColumns block. Instead of processing the entire 128-bit state in one large combinational block, it processes one 32-bit AES column at a time. It moves through LOAD_COL0 to LOAD_COL3 and MIX_COL0 to MIX_COL3, then asserts the done signal.

The second is the sequential key expansion block. It performs RotWord, SubWord, RCON addition, and then generates the four 32-bit words of the next round key.

Both FSMs help reduce hardware activity by breaking the operation into controlled sequential steps. This improves area efficiency and reduces switching activity.

Results

Now I will discuss the synthesis results.

The baseline AES design had a total cell count of 83,352. The proposed design reduced this to 2,694 cells. This is a major reduction in hardware usage.

The total area reduced from 202,949.64 square micrometers to 11,841.48 square micrometers. This gives around 17.1 times smaller area compared to the baseline.

For power, the total power reduced from 2.19×10^{-2} W to 8.53×10^{-4} W. This shows that datapath reuse significantly reduces power consumption.

In the power distribution graph, the baseline design has a high power share in logic because many combinational blocks are active. In the proposed design, logic power is reduced, while the register share becomes higher. This is expected because the architecture uses sequential reuse and stores intermediate states.

The timing analysis also confirms that the design meets the timing requirement. The target clock period is 10 ns, which corresponds to 100 MHz. The data path delay is 3102 ps, and the setup slack is positive, around 6806 ps. So the design is timing-clean after synthesis.

Simulation Verification

Functional verification was performed using simulation waveforms.

The AES module receives a 128-bit plaintext and a 128-bit key. After the encryption sequence completes, the ciphertext output is generated and the done signal is asserted.

The waveform confirms correct encryption behavior for AES-128 known-answer tests. This verifies that even though the hardware is reused sequentially, the encryption result remains correct.

Trade-Off

The main trade-off in this design is throughput versus area and power.

A fully pipelined AES architecture can give very high throughput, but it requires more hardware and consumes more power. Our proposed architecture has higher latency because the rounds execute sequentially. However, it is much smaller and more power-efficient.

For IoT systems, this trade-off is acceptable because many devices do not need maximum encryption throughput. They need reliable encryption with low area and low energy consumption.

Applications

This design is suitable for secure embedded and IoT applications.

For example, in healthcare wearable devices, sensor data such as heart rate, temperature, or patient monitoring values can be encrypted before wireless transmission.

It can also be used in smart meters, industrial monitoring nodes, environmental sensors, and low-power edge devices where secure data transmission is required.

Because the design uses less area and power, it can be integrated into small FPGA or ASIC-based security accelerators.

Future Work

As future work, this AES core can be integrated into a lightweight IoT security processor based on RISC-V.

The future system includes a 5-stage RV32I-style processor, AES-128 accelerator, AES-CTR mode, sensor MMIO, UART transmitter, interrupt controller, DMA-lite engine, power activity counters, and a custom security ISA.

In that system, sensor input will be encrypted using AES and transmitted through UART. UART can also be used for verification by printing the given input and the decrypted or encrypted output.

Compared to traditional software AES execution, custom hardware acceleration can reduce execution time and processor workload. Compared to conventional MMIO-only control, custom ISA instructions can reduce control overhead and improve integration between processor and accelerator.

This makes the design useful for practical low-power IoT security systems.

Conclusion

To conclude, this work presents a low-power iterative AES-128 architecture based on hardware reuse.

The proposed design reduces duplicate AES round logic and minimizes switching activity by activating only the required datapath blocks under FSM control.

Synthesis results show major improvement in area and power compared to the baseline AES architecture. The design also meets timing and produces correct AES encryption output.

Therefore, the proposed architecture is suitable for FPGA-based embedded and IoT security applications where compact area and low power are more important than maximum throughput.

Thank you. I will be happy to answer your questions.

Waveform Clock-Cycle Explanation

Use this part while explaining the simulation waveform. The exact cycle count may change slightly depending on the RTL control implementation, but the waveform should be explained using the start, round, done, and ciphertext transitions.

Cycle 0: Reset is active. Internal state registers, round counter, done signal, and output ciphertext register are cleared.

Cycle 1: Reset is released. Plaintext and key are stable at the input. The AES core remains idle until start is asserted.

Cycle 2: Start is asserted for one clock cycle. The FSM samples the plaintext and key and begins the encryption operation.

Cycle 3: Initial AddRoundKey is performed. The plaintext P is XORed with the initial key K0 to form $S0 = P \text{ XOR } K0$.

Rounds 1 to 9: For each normal AES round, the same datapath is reused sequentially. SubBytes, ShiftRows, MixColumns, Key Expansion, and AddRoundKey are activated under FSM control. Only the required block is active in each step, which reduces switching activity.

MixColumns internal cycles: The sequential MixColumns unit processes one 32-bit column at a time: column 0, column 1, column 2, and column 3. After the fourth column, the MixColumns done signal is asserted.

Key expansion cycle: The key expansion block performs RotWord, SubWord, RCON addition, and generates the next 128-bit round key. This avoids storing all round keys separately.

Final round: The final AES round performs SubBytes, ShiftRows, and AddRoundKey only. MixColumns is skipped according to the AES-128 standard.

Test end: The test ends when the AES core asserts done = 1. At this clock edge, ciphertext is valid and is compared with the expected AES-128 known-answer value.

Pass condition: If ciphertext matches the expected value when done is high, the testbench reports the test as passed. If it does not match, the testbench reports a mismatch or failure.

Suggested spoken line: In the waveform, the important observation is that the inputs remain stable when start is sampled, the FSM progresses through iterative AES rounds, and the ciphertext is checked only after done becomes high. This confirms correct multi-cycle encryption behavior.

NIST Test Vector Explanation

NIST stands for the National Institute of Standards and Technology. It is the U.S. standards body that published AES as a federal information processing standard. AES is defined in FIPS 197, and NIST also provides standard known-answer test vectors for checking whether an AES implementation is correct.

A NIST test vector is a fixed set of plaintext, key, and expected ciphertext. For AES-128, the key is 128 bits, the plaintext block is 128 bits, and the correct ciphertext is already known from the standard reference algorithm. These vectors are useful because they allow us to verify that our RTL implementation is not only producing an output, but producing the exact AES-standard output.

In our simulation, the testbench applies AES-128 known-answer vectors. For each vector, the plaintext and key are given to the AES core, the start signal begins encryption, and after the done signal goes high, the produced ciphertext is compared with the expected NIST ciphertext.

The test ends for each vector when done = 1 and the ciphertext comparison is completed. If the generated ciphertext matches the expected value, that test vector passes. If any bit differs, the testbench flags a failure. Passing these vectors confirms functional correctness of the AES datapath, round control, key expansion, and final-round behavior.

Example spoken line: Here, NIST known-answer vectors are used as our test cases. That means for every plaintext and key pair, the expected ciphertext is already defined by the AES standard. Our waveform verifies that when done becomes high, the RTL output matches that expected ciphertext.

If asked why NIST vectors matter: They are trusted reference vectors, so passing them gives confidence that the hardware implementation follows the AES-128 standard correctly, independent of our own test assumptions.

General Research Framing

1. What is the core contribution of your work?

Answer: A low-power AES-128 architecture that reuses one datapath iteratively instead of duplicating round hardware, reducing switching activity, area, and total power while preserving correct AES encryption.

2. Why is this problem important for IoT applications?

Answer: IoT nodes usually have limited energy, area, and thermal budget. A compact AES accelerator enables secure transmission without the overhead of fully parallel or software-only encryption.

3. What is the novelty compared with standard AES hardware?

Answer: The novelty is architectural activity reduction through controlled datapath reuse and sequential FSM operation, targeting the activity factor in dynamic power rather than only optimizing gates after synthesis.

4. Is AES itself modified in your design?

Answer: No. The AES-128 algorithm remains standard. Only the hardware architecture and execution schedule are changed.

5. What is the main trade-off?

Answer: The design trades peak throughput for lower area and power. It is suitable where energy efficiency is more important than maximum encryption rate.

6. Why did you choose AES-128 and not AES-192 or AES-256?

Answer: AES-128 is widely used in embedded security, has lower key schedule complexity, and is sufficient for many IoT applications while keeping hardware compact.

7. How do you define hardware reuse in this work?

Answer: The same SubBytes, ShiftRows, MixColumns, AddRoundKey, and Key Expansion resources are reused across rounds instead of instantiating multiple parallel round units.

8. What does iterative architecture mean?

Answer: It means the AES rounds are executed sequentially over multiple clock cycles using a shared datapath and control FSM.

9. Why not use a fully pipelined AES?

Answer: Fully pipelined AES provides high throughput but consumes more area and switching power. Our target is low-power IoT, where throughput demand is moderate.

10. What is your baseline design?

Answer: The baseline is a more parallel AES implementation synthesized under the same tool and technology conditions for fair comparison.

AES Algorithm

11. Explain AES-128 in one minute.

Answer: AES-128 encrypts a 128-bit plaintext block using a 128-bit key. It performs an initial AddRoundKey, then 10 rounds of substitution, permutation, mixing, and key addition, with MixColumns omitted in the final round.

12. What is SubBytes?

Answer: SubBytes is a nonlinear byte substitution using the Rijndael S-box. It provides confusion and protects against linear cryptanalysis.

13. What is ShiftRows?

Answer: ShiftRows cyclically shifts the rows of the AES state matrix to spread byte positions across columns.

14. What is MixColumns?

Answer: MixColumns performs finite-field matrix multiplication over $GF(2^8)$, providing diffusion across each 32-bit column.

15. Why is MixColumns skipped in the final round?

Answer: That is defined by the AES standard. The final round performs SubBytes, ShiftRows, and AddRoundKey only.

16. What is AddRoundKey?

Answer: It XORs the AES state with the current round key. It is the step where key material directly affects the state.

17. What is key expansion?

Answer: Key expansion generates round keys from the original 128-bit cipher key using RotWord, SubWord, RCON, and XOR operations.

18. What is the AES state representation?

Answer: The 128-bit block is treated as a 4 by 4 byte matrix, called the AES state.

19. Why is AES suitable for hardware?

Answer: Its operations are byte substitution, XOR, shifts, and finite-field arithmetic, which map well to combinational and sequential digital logic.

20. Does reducing hardware change security?

Answer: No, not if the AES transformation sequence remains standard and functionally correct. Security is algorithmic; hardware reuse affects performance and implementation cost.

RTL and Architecture

21. What are the main RTL modules?

Answer: The main modules include AES control FSM, SubBytes/S-box, ShiftRows, sequential MixColumns, Key Expansion, state registers, and output control.

22. How does the FSM reduce power?

Answer: It activates only the required operation in each cycle, reducing simultaneous switching in unused logic blocks.

23. Why process MixColumns sequentially?

Answer: Processing one 32-bit column at a time reduces combinational logic width and switching capacitance.

24. What happens when start is asserted?

Answer: The FSM samples plaintext and key, performs initial AddRoundKey, then begins iterative round execution.

25. When is done asserted?

Answer: Done is asserted after the final round output is valid and ciphertext has been written to the output register.

26. What is the role of the round counter?

Answer: It tracks the AES round number and controls whether normal rounds or the final round should be executed.

27. How do you avoid unnecessary toggling?

Answer: By using FSM gating, sequential execution, and controlled register updates so inactive blocks are not repeatedly stimulated.

28. Why is register share higher in the proposed power distribution?

Answer: Because combinational logic is reduced significantly, so sequential storage becomes a larger percentage of the remaining power.

29. Does the design use clock gating?

Answer: The current focus is architectural reuse and controlled state transitions. Clock gating can be added later for further idle-power reduction.

30. What is the critical path likely to include?

Answer: It may include S-box lookup, XOR network, MixColumns finite-field operations, or key expansion logic depending on synthesis mapping.

VLSI / Semiconductor Devices

31. Why does reducing area reduce power?

Answer: Lower area usually means fewer cells, lower capacitance, and fewer internal nodes switching, which reduces dynamic and internal power.

32. Explain dynamic power in CMOS.

Answer: Dynamic power is $P_{\text{dyn}} = \alpha CL V^2 f$, where α is switching activity, CL is load capacitance, V is supply voltage, and f is clock frequency.

33. Which term did your project mainly reduce?

Answer: The project mainly reduces α , the switching activity factor, and also reduces effective capacitance by reducing cell count and logic duplication.

34. Why is switching power important in AES?

Answer: AES has many XORs, S-box accesses, and data-dependent transitions. Parallel datapaths increase toggling and therefore dynamic power.

35. What is leakage power?

Answer: Leakage is static power consumed even when transistors are not switching, mainly due to subthreshold leakage and gate leakage.

36. Why did leakage reduce in your proposed design?

Answer: The smaller cell count and area reduce the number of transistors contributing leakage.

37. What is internal power?

Answer: Internal power is consumed inside standard cells during transitions, including short-circuit current and charging internal capacitances.

38. What is switching power?

Answer: Switching power is consumed by charging and discharging interconnect and load capacitances at signal nets.

39. What does positive setup slack mean?

Answer: It means the data arrives before the required setup time, so the design meets the target clock constraint.

40. If voltage is reduced, what happens to power?

Answer: Dynamic power reduces quadratically with voltage, but timing delay usually increases, so voltage scaling must be balanced with timing closure.

FPGA and Implementation

41. Why target FPGA-based IoT systems?

Answer: FPGAs allow rapid prototyping of secure accelerators and are used in edge systems where custom hardware can improve energy efficiency.

42. How is this architecture FPGA-friendly?

Answer: It uses fewer replicated logic blocks and a controlled FSM datapath, reducing LUT/register pressure compared with unrolled AES.

43. Would block RAM be useful for S-box implementation?

Answer: Yes. The S-box can be implemented using LUT logic or memory resources, depending on area, speed, and power trade-offs.

44. What synthesis tool was used?

Answer: The poster reports synthesis using Cadence Genus with a TSMC 55 nm low-power standard-cell library under typical conditions.

45. Can ASIC results directly predict FPGA power?

Answer: Not exactly. ASIC and FPGA fabrics differ, but architectural trends such as reduced logic duplication and switching are still meaningful.

46. How would you validate this on FPGA?

Answer: By synthesizing in Quartus or Vivado, checking resource utilization, timing, and measuring or estimating power using switching activity from simulation.

47. What is the target clock?

Answer: The poster uses a 10 ns target clock period, corresponding to 100 MHz.

48. What does a 3.102 ns data path delay imply under a 10 ns clock?

Answer: It means the combinational path is much faster than required, giving positive timing margin.

49. Why is the design not throughput optimized?

Answer: Because it intentionally serializes operations to reduce area and power. Throughput is secondary for the intended IoT use case.

50. How can throughput be improved later?

Answer: By partially unrolling selected stages, duplicating only bottleneck modules, or adding lightweight pipelining while preserving controlled activation.

Data Analysis / Results

51. What are your strongest numerical results?

Answer: Cell count reduced from 83,352 to 2,694, area from 202,949.64 to 11,841.48 μm^2 , and total power from 2.19e-2 W to 8.53e-4 W.

52. What is the area reduction factor?

Answer: Approximately 17.1x smaller area, calculated as 202,949.64 divided by 11,841.48.

53. What is the cell-count reduction factor?

Answer: Approximately 30.9x fewer cells, calculated as 83,352 divided by 2,694.

54. What is the total power reduction factor?

Answer: Approximately 25.7x reduction, calculated as 2.19e-2 divided by 8.53e-4.

55. Why should the comparison be under the same synthesis environment?

Answer: Same tool, library, voltage, temperature, and constraints make the comparison fair by isolating architecture as the main variable.

56. What is the meaning of 70.86 percent register power share?

Answer: It means registers contribute the largest portion of the proposed design's remaining power after combinational logic is greatly reduced.

57. Why did logic share reduce from 75.89 to 22.50 percent?

Answer: The proposed design removes duplicated combinational AES round logic and uses one reused datapath.

58. How do you interpret the timing slack result?

Answer: Positive slack confirms the design can operate at the target 100 MHz clock with timing margin.

59. What result best supports your research claim?

Answer: The combined reduction in cell count, area, logic power share, and total power supports the claim that datapath reuse improves low-power efficiency.

60. What additional result would strengthen the paper?

Answer: A normalized comparison including throughput, energy per block, and FPGA resource/power measurements would strengthen the evaluation.

Verification / NIST

61. What is NIST?

Answer: NIST is the National Institute of Standards and Technology, which standardized AES in FIPS 197 and provides reference material for validating AES implementations.

62. What is a NIST test vector?

Answer: It is a known plaintext, key, and expected ciphertext combination used to verify that an AES implementation produces the standard-correct result.

63. Why use known-answer tests?

Answer: They give deterministic pass/fail validation because the expected ciphertext is already known.

64. When does a simulation test end?

Answer: A test ends when done becomes high and the output ciphertext is compared against the expected ciphertext.

65. What happens at reset?

Answer: Internal registers, counters, output, and done signal are cleared.

66. What happens when start goes high?

Answer: The FSM samples valid inputs and begins the encryption sequence.

67. How do you know encryption is correct?

Answer: The generated ciphertext matches the expected NIST known-answer ciphertext when done is asserted.

68. What if ciphertext mismatches by one bit?

Answer: The test fails because cryptographic correctness requires exact bit-level agreement.

69. What signals are important in the waveform?

Answer: Clock, reset, start, plaintext, key, round/state indicators, ciphertext, and done.

70. Does passing one NIST vector prove full correctness?

Answer: It provides strong evidence for that case, but multiple vectors and corner cases are needed for broader verification.

Cyber Security

71. Does low-power AES reduce cryptographic security?

Answer: No, if the AES standard operations are preserved exactly. It changes implementation efficiency, not the AES algorithm.

72. What security threat is relevant to hardware AES?

Answer: Side-channel attacks, especially power analysis, are relevant because switching activity can leak data-dependent information.

73. Does your design prevent side-channel attacks?

Answer: Not fully. It reduces switching activity, but side-channel resistance requires dedicated countermeasures such as masking, hiding, balancing, or randomization.

74. Can lower switching activity help side-channel behavior?

Answer: It may reduce total activity, but it does not automatically remove data-dependent leakage.

75. What is AES-CTR mode and why mention it in future work?

Answer: AES-CTR turns AES into a stream-like mode by encrypting counters and XORing with data, which is useful for sensor data streams.

76. Is ECB mode recommended for real IoT data?

Answer: ECB is useful for primitive verification but not ideal for real repeated data patterns. CTR, CBC, or GCM-like authenticated modes are better depending on requirements.

77. What is the difference between encryption and authentication?

Answer: Encryption provides confidentiality; authentication verifies integrity and source. AES alone does not always provide authentication.

78. What would you add for stronger IoT security?

Answer: Authenticated encryption, secure key storage, random nonce management, side-channel protection, and secure boot integration.

79. How should keys be protected in hardware?

Answer: Keys should be stored in protected registers or secure memory with limited access, zeroization, and controlled debug visibility.

80. What is the risk of UART output in verification?

Answer: UART is fine for lab verification, but real systems must avoid exposing plaintext keys or sensitive decrypted data over unsecured debug channels.

Thermal / Mechanical

81. How does power reduction affect thermal behavior?

Answer: Lower power reduces heat generation, which improves thermal reliability and helps compact embedded systems operate without heavy cooling.

82. Why would a mechanical or thermal expert care about AES hardware power?

Answer: In dense IoT or edge devices, every milliwatt affects temperature rise, enclosure design, battery life, and reliability.

83. What is the relation between dynamic power and heat?

Answer: Electrical power consumed by CMOS circuits is ultimately dissipated as heat, so reducing dynamic power reduces thermal load.

84. How can thermal issues affect digital circuits?

Answer: Higher temperature increases leakage, may reduce timing margin, and can accelerate aging mechanisms.

85. Does smaller area always reduce temperature?

Answer: Not always. Smaller area can increase power density, but in this design total power drops significantly, so thermal load is expected to improve.

Electrical / Circuits

86. What is setup slack?

Answer: Setup slack is the difference between required arrival time and actual data arrival time at a register. Positive slack means timing is met.

87. What is clock frequency for 10 ns period?

Answer: $f_{clk} = 1 / 10 \text{ ns} = 100 \text{ MHz}$.

88. What causes glitches in combinational logic?

Answer: Different path delays can cause temporary transitions before signals settle, increasing switching power.

89. How does sequential operation reduce glitches?

Answer: By reducing large concurrent combinational networks and registering intermediate values, fewer wide datapaths switch simultaneously.

90. What is capacitance in the power equation?

Answer: It is the effective load capacitance of gates and interconnects that must be charged and discharged during switching.

91. What would happen if frequency doubles?

Answer: Dynamic power approximately doubles, assuming activity, capacitance, and voltage remain constant.

92. What would happen if voltage scales from 1.0 V to 0.8 V?

Answer: Dynamic power scales by V^2 , so ideal dynamic power becomes 0.64 of the original, but delay increases.

93. What is the role of FSM in circuit-level control?

Answer: The FSM sequences operations and controls register enables and start/done handshakes, preventing uncontrolled datapath activity.

94. Why use registers between operations?

Answer: Registers store intermediate states, isolate timing paths, and allow multi-cycle operation with controlled data movement.

95. What are asynchronous reset concerns?

Answer: Asynchronous reset must be released carefully to avoid metastability or inconsistent state recovery across registers.

Defense / Critical Questions

96. What is the biggest limitation of your design?

Answer: Higher latency compared with pipelined AES. The design is optimized for area and power, not maximum throughput.

97. Can you claim it is the first ever implementation?

Answer: A careful claim is that this work proposes and evaluates an activity-aware iterative reusable architecture for the stated FPGA/IoT context; absolute first-ever claims need exhaustive literature proof.

98. Why are references limited in the poster?

Answer: Poster space is limited, so the most relevant and recent IEEE/journal works are selected, while the paper can include the full bibliography.

99. How would you calculate energy per encryption?

Answer: Energy per encryption is $E_{enc} = P_{avg} \times T_{enc}$, where T_{enc} depends on the number of cycles and clock period.

100. What if an application needs high throughput?

Answer: Then a partially unrolled or pipelined AES may be more suitable. Our design targets low-power moderate-throughput nodes.

Rapid Revision Numbers

Metric	Value / Interpretation
Target clock	10 ns = 100 MHz
Cell count	83,352 to 2,694, about 30.9x reduction
Area	202,949.64 μm^2 to 11,841.48 μm^2 , about 17.1x smaller
Total power	2.19×10^{-2} W to 8.53×10^{-4} W, about 25.7x reduction
Timing	Data path delay 3102 ps, setup slack about +6806 ps
Main reduced parameter	Switching activity alpha and effective logic capacitance
Verification	NIST AES-128 known-answer vectors; pass when ciphertext matches at done = 1
Best defense line	Standard AES function, reused datapath hardware, lower active logic per cycle

Clock-Cycle / Waveform Explanation

Cycle / phase	What happens	What to point out in waveform
Reset phase	reset = 1 clears state, round counter, ciphertext, and done.	Outputs are initialized.
Idle phase	reset is released; plaintext and key remain stable.	Core waits for start.
Start cycle	start = 1 is sampled by the FSM.	Encryption sequence begins.
Initial operation	Initial AddRoundKey forms $S_0 = P \text{ XOR } K_0$.	State changes after start.
Rounds 1-9	Same datapath executes SubBytes, ShiftRows, MixColumns, KeyExpansion, and AddRoundKey sequentially.	Iterative multi-cycle behavior is visible.
MixColumns internal cycles	AES_MIXCOL_SEQ processes 32-bit columns one by one: col 0 to col 3.	Column sequencing proves hardware reuse.
Final round	SubBytes, ShiftRows, and AddRoundKey execute; MixColumns is skipped.	Final AES-standard behavior.
Test end	done = 1; ciphertext is valid and compared with expected NIST ciphertext.	Pass condition occurs only at done.

Exact Proposed-Design Results from Reports

Metric	Exact report value
Cell count	2,694
Cell area	11841.480 μm^2
Net area	0.000 μm^2
Total area	11841.480 μm^2
Leaf instance count	2,694
Sequential instance count	792
Combinational instance count	1,902
Hierarchical instance count	8
Total power	8.53125e-04 W
Leakage power	4.51803e-07 W
Internal power	6.96687e-04 W
Switching power	1.55986e-04 W

Exact Power Distribution from report_power.rpt

Category	Leakage (W)	Internal (W)	Switching (W)	Total (W)	Share
Register	2.57437e-07	5.74447e-04	2.98249e-05	6.04530e-04	70.86%
Logic	1.94366e-07	1.22239e-04	6.95614e-05	1.91995e-04	22.50%
Clock	0.00000e+00	0.00000e+00	5.66000e-05	5.66000e-05	6.63%
Subtotal	4.51803e-07	6.96687e-04	1.55986e-04	8.53125e-04	99.99%
Percentage	0.05%	81.66%	18.28%	100.00%	100.00%

Exact Timing Backup

Timing item	Exact value
Target clock period	10000.0 ps = 10 ns
Clock frequency	100 MHz
Required time	9908 ps
Setup time	82 ps
Uncertainty	10 ps
Data path delay	3102 ps
Setup slack	6806 ps
QoR path slack	6806.5 ps
TNS	0.0
Violating paths	0
Timing status	MET

Timing defense: Since slack is positive and violating paths are zero, the proposed AES design meets the 10 ns timing constraint.

9. Comparison and Reduction Calculations

Metric	Baseline AES	Proposed AES	Reduction interpretation /
Cell count	83,352	2,694	30.9x fewer cells
Total area	202,949.64 um^2	11,841.480 um^2	17.1x smaller area
Total power	2.19e-02 W	8.53125e-04 W	25.7x lower power

Area reduction calculation: $202,949.64 / 11,841.480 = 17.14x$.

Cell-count reduction calculation: $83,352 / 2,694 = 30.94x$.

Power reduction calculation: $2.19e-02 / 8.53125e-04 = 25.67x$.

State	Condition	Action	Next State
S_IDLE	start=0	busy=0, done=0	S_IDLE
S_IDLE	start=1	state_reg=plaintext^key, round_key=key, round=1, busy=1	S_SUB_START
S_SUB_START	—	sbox_start=1	S_SUB_WAIT
S_SUB_WAIT	sbox_done=0	wait	S_SUB_WAIT
S_SUB_WAIT	sbox_done=1 and round<10	state_reg=ShiftRows(sb_out)	S_MIX_START
S_SUB_WAIT	sbox_done=1 and round=10	state_reg=ShiftRows(sb_out)	S_KEY_START
S_MIX_START	—	mix_start=1	S_MIX_WAIT
S_MIX_WAIT	mix_done=0	wait	S_MIX_WAIT
S_MIX_WAIT	mix_done=1	state_reg=mix_out	S_KEY_START
S_KEY_START	—	key_start=1	S_KEY_WAIT
S_KEY_WAIT	key_done=0	wait	S_KEY_WAIT
S_KEY_WAIT	key_done=1 and round<10	round_key=new_key, state_reg=state_reg^new_key, round++	S_SUB_START
S_KEY_WAIT	key_done=1 and round=10	ciphertext=state_reg^new_key	S_DONE
S_DONE	—	done=1, busy=0	S_IDLE

- Ciphertext becomes valid: around cycle 285
- `done` becomes high: cycle 286
- For presentation/interview, say: "The encryption latency is 286 clock cycles per 128-bit block, measured until `done` is asserted."

At your target clock:

$$T_{clk} = 10ns, \quad f_{clk} = 100MHz$$

Encryption time:

$$286 \times 10ns = 2860ns = 2.86\mu s$$

Throughput:

$$\begin{aligned} \text{Throughput} &= \frac{128 \times 100MHz}{286} \\ &= 44.76 \text{ Mbps} \end{aligned}$$